Taller 2 Corte: React

Estudiante: Diego Salcedo Colon
Código: T00077067
Curso: Desarrollo Frontend · UTB
Fecha: 16 de octubre de 2026

Punto 1 · ¿Qué es React?

¿Qué es React y para qué sirve dentro del desarrollo web?

React es una librería de JavaScript que se usa para construir interfaces de usuario, es decir, lo que el usuario ve y con lo que interactúa. La idea principal es armar la interfaz con diferentes piezas llamadas componentes. Se usa en la web y también en el móvil. [1][4]

¿Quién lo creó y en qué año apareció?

React fue creado por Jordan Walke, un ingeniero de Facebook. Empezó a usarse dentro de Facebook en 2011 y se publicó como código abierto en mayo de 2013. [6][7]

¿Es una librería o un framework? Explica la diferencia y justifica tu respuesta

React es una librería porque se enfoca en la interfaz y no trae por sí sola enrutamiento ni obtención de datos. Una librería es un conjunto de herramientas que uno llama cuando las necesita, mientras que un framework trae una estructura completa y define cómo organizar la aplicación; en cierto modo, el framework manda. [2]

¿Qué significa que React sea declarativo?

Que React sea declarativo significa que describes cómo debe verse la interfaz para un estado dado, y React se encarga de actualizar la pantalla cuando ese estado cambia. [1]

Punto 2 · ¿Qué problema resuelve React?

¿Cuál era la dificultad principal de construir interfaces con JavaScript "a mano" (manipulando el DOM)?

Con JavaScript hay que buscar cada elemento de la página y modificarlo cada vez que cambian los datos. En una aplicación esto se vuelve difícil de seguir, ya que muchos lugares del código tocan el mismo elemento y es fácil olvidar actualizarlo o arreglarlo. [1]

Diferencia entre el enfoque imperativo y el enfoque declarativo

Imperativo: le dices al navegador los pasos para llegar al resultado.

Declarativo: describes el resultado que quieres para cada estado y la librería calcula los pasos. [1]

Ejemplo imperativo:

```
<p id="contador">0</p>
<button onclick="sumar()">Sumar</button>
```

```
<script>
  function sumar() {
    const el = document.getElementById("contador");
    el.textContent = Number(el.textContent) + 1;
  }
</script>
```

Qué hace el código:

- `<p id="contador">0</p>`: un párrafo que muestra el número, con un id para poder encontrarlo.
- `<button onclick="sumar()">`: al hacer clic se ejecuta la función sumar.
- `document.getElementById("contador")`: busca el párrafo en la página.
- `Number(el.textContent) + 1;`: lee el texto actual, lo convierte a número y le suma 1.
- `el.textContent`: escribe el nuevo valor directamente en el párrafo.

Por qué es imperativo: se le dice al navegador cada paso: buscar el elemento, leerlo, calcular y modificarlo. Si el contador se mostrara en 5 lugares, tendría que acordarme de actualizar los 5.

Ejemplo declarativo:

```
import { useState } from "react";

function Contador() {
  const [cuenta, setCuenta] = useState(0);

  return (
    <div>
      <p>Clics: {cuenta}</p>
      <button onClick={() => setCuenta(cuenta + 1)}>Sumar</button>
    </div>
  );
}
```

Qué hace el código:

- `useState(0)`: crea un dato llamado cuenta que empieza en 0. Devuelve el valor y una función para cambiarlo.
- `<p>Clics: {cuenta}</p>`: describe que el párrafo muestra el valor actual de cuenta. Las llaves {} insertan la variable.
- `onClick={() => setCuenta(cuenta + 1)}`: al hacer clic, pide cambiar el estado a cuenta + 1.

Por qué es declarativo: no busco ningún elemento ni modifico el texto. Solo describo qué se ve para cada valor de cuenta y React se encarga de actualizar la pantalla.

¿Cómo ayuda React a mantener la interfaz sincronizada con los datos (el estado)?

En React la interfaz depende del estado: cuando el estado cambia, React vuelve a calcular qué debe mostrarse, lo compara con lo anterior y actualiza en el DOM solo lo que cambió. Así no tienes que recordar qué partes de la pantalla actualizar cada vez que cambia un dato. [1]

¿Qué rol juegan los componentes reutilizables en la solución de ese problema?

Cada componente encapsula su propio pedazo de interfaz y su lógica. Se puede reutilizar en varios lugares y, si hay que cambiarlo, se modifica en un solo sitio. [1]

Punto 3 · Principales componentes de React

3.1 Componentes

Definición: una función de JavaScript que devuelve el fragmento de interfaz que se debe mostrar. [1]

Características principales:

- El nombre empieza con mayúscula.
- Se usan como etiquetas.
- Se pueden anidar y reutilizar.

Ejemplo de código:

```
function Saludo() {
  return <h1>Hola, mundo</h1>;
}

export default function App() {
  return (
    <div>
      <Saludo />
      <Saludo />
    </div>
  );
}
```

Qué hace:

- `function Saludo()`: crea un componente.
- Devuelve un título <h1>.
- `export default function App()`: es el componente principal.
- <Saludo /> aparece dos veces dentro de App: el mismo componente se reutiliza y se muestran dos títulos.

3.2 JSX

Definición: una sintaxis que permite escribir marcado parecido a HTML dentro del código JavaScript. El navegador no lo entiende directamente: una herramienta de build lo convierte a JavaScript normal. [1]

Características principales:

- Un solo elemento raíz.
- Todas las etiquetas se cierran.
- Los atributos van en camelCase.
- Se escribe JavaScript dentro de llaves.

Ejemplo de código:

```
const nombre = "Diego";
const elemento = <h2 className="titulo">Hola, {nombre}</h2>;
```

Qué hace:

- `const nombre = "Diego"`: guarda un texto en una variable.
- <h2 className="titulo">: crea un título y le asigna la clase CSS titulo.

- `{nombre}`: inserta el valor de la variable dentro del texto, así que se ve "Hola, Diego".

3.3 Props

Definición: los datos que un componente padre le pasa a un componente hijo, como si fueran argumentos de una función. [1]

Características principales:

- Son de solo lectura (el hijo no las modifica).
- Permiten reutilizar un mismo componente con datos distintos.

Ejemplo de código:

```
function Producto({ nombre, precio }) {
  return <p>{nombre}: {precio} USD</p>;
}

// Uso
<Producto nombre="Camiseta" precio={30} />
<Producto nombre="Gorra" precio={15} />
```

Qué hace:

- `{ nombre, precio }`: el componente recibe dos props.
- `<p>{nombre}: {precio} USD</p>`: las muestra en pantalla.
- Las dos líneas de uso llaman al mismo componente con datos diferentes: se muestran "Camiseta: 30 USD" y "Gorra: 15 USD".
- Los textos van entre comillas ("Camiseta") y los números entre llaves ({30}).

3.4 Estado (useState)

Definición: la "memoria" de un componente: datos que pueden cambiar con el tiempo (un contador, el texto de un input). `useState` es el Hook que permite declararlo. [1]

Características principales:

- `useState(valorInicial)` devuelve un par: el valor actual y una función para cambiarlo.
- Al llamar esa función, React vuelve a renderizar el componente y la interfaz se actualiza sola.

Ejemplo de código:

```
import { useState } from "react";

function Contador() {
  const [cuenta, setCuenta] = useState(0);

  return (
    <button onClick={() => setCuenta(cuenta + 1)}>
      Clics: {cuenta}
    </button>
  );
}
```

Qué hace:

- `import { useState } from "react"`: trae el Hook desde React.

- `useState(0)`: crea el estado cuenta con valor inicial 0.
- `setCuenta`: la función para cambiar ese estado.
- `onClick={() => setCuenta(cuenta + 1)}`: en cada clic, cambia el estado a cuenta + 1.
- Al cambiar el estado, React vuelve a dibujar el botón y el número se actualiza.

3.5 Eventos (onClick, onChange)

Definición: las acciones del usuario (clic, escribir, enviar un formulario). En React se manejan pasando una función a atributos como `onClick` y `onChange`. [1]

Características principales:

- Los nombres van en camelCase.
- Se pasa la función, no el resultado de llamarla.
- La función recibe un objeto de evento (e) con información de lo que pasó.

Ejemplo de código:

```
import { useState } from "react";

function Formulario() {
  const [nombre, setNombre] = useState("");

  return (
    <>
      <input value={nombre} onChange={(e) => setNombre(e.target.value)} />
      <p>Hola, {nombre}</p>
    </>
  );
}
```

Qué hace:

- `useState("")`: crea el estado nombre, que empieza vacío.
- `value={nombre}`: el input siempre muestra lo que hay en el estado.
- `onChange={(e) => setNombre(e.target.value)}`: cada vez que el usuario escribe, `e.target.value` es el texto actual del input y se guarda en el estado.
- `<p>Hola, {nombre}</p>`: se actualiza en vivo mientras escribes.
- `<>...</>`: un fragmento, para devolver dos elementos sin un contenedor extra.

Punto 4 · Importancia de React en la industria

¿Por qué tantas empresas eligen React para sus productos web?

- Es maduro: lleva más de una década desde su lanzamiento público (2013).
- Se basa en componentes reutilizables, lo que facilita mantener productos grandes.
- Funciona en web y también en móvil (React Native), así que un equipo reutiliza lo que sabe.
- Meta reporta que más de cinco mil personas construyen productos con React cada mes, en web, móvil, escritorio y realidad mixta. [4]

¿Qué aporta React al ecosistema de desarrollo (empaquetadores, frameworks basados en React, etc.)?

React solo cubre la interfaz, así que alrededor de él hay herramientas y frameworks que resuelven lo demás: enrutamiento, obtención de datos y generación de HTML. La documentación oficial recomienda frameworks como Next.js, React Router y Expo para empezar proyectos completos. React Native lleva el mismo enfoque a apps móviles. [2][4]

¿Qué papel juegan su comunidad y su mantenimiento? ¿Quién lo mantiene hoy?

- Durante años lo mantuvo un equipo de Meta con ayuda de la comunidad de código abierto. [6]
- Desde el **24 de febrero de 2026**, React, React Native y JSX ya no son propiedad de Meta: pasaron a la **React Foundation**, una fundación independiente alojada en la Linux Foundation. [3][8]
- Entre sus miembros fundadores están Amazon, Microsoft, Meta, Vercel, Expo, Callstack, Software Mansion y Huawei, y su director ejecutivo es Seth Webster. [8]
- La dirección técnica de React sigue siendo independiente de la junta de la fundación. [3]

¿Por qué es una habilidad relevante para un desarrollador front-end hoy en día?

- Es una de las tecnologías web más utilizadas, según la Stack Overflow Developer Survey 2025, así que conocerlo abre oportunidades en muchos equipos y proyectos. [9]
- Lo usan empresas grandes (Meta, Netflix; ver Punto 5).
- Aprender React enseña ideas que se repiten en otras herramientas: componentes, estado y renderizado declarativo.

Punto 5 · Grandes empresas que utilizan React

Empresa 1: Meta (Facebook e Instagram)

En qué producto o sección lo usan:

- En la experiencia de escritorio de Facebook.com, donde reutilizan funciones como el News Feed y los comentarios. [4]
- En las versiones de Facebook e Instagram para Meta Quest (realidad mixta), hechas con React Native. [4]
- Un ingeniero que trabajó en la web de Instagram explicó en un podcast que allí también son grandes usuarios de React. [10]

Por qué es un caso relevante:

Meta es la empresa donde nació React, así que es el caso de uso a mayor escala y el que más influye en cómo evoluciona la librería. Además, reporta que más de cinco mil personas construyen productos con React cada mes, en web, móvil, escritorio y realidad mixta. [4]

Fuentes:

- [4] Engineering at Meta, "React at Meta Connect 2024": https://engineering.fb.com/2024/10/02/android/react-at-meta-connect-2024/
- [10] JavaScript Jabber 073, con Pete Hunt y Jordan Walke: https://topenddevs.com/podcasts/javascript-jabber/073-jsj-react-with-pete-hunt-and-jordan-walke

Empresa 2: Netflix

En qué producto o sección lo usan:

- En Netflix.com, donde React se usa para generar el HTML de las páginas en el servidor. [5]
- Un caso de estudio sobre su rendimiento explica que quitaron React del lado del cliente en la página de inicio para usuarios sin sesión, porque así la página se volvía interactiva más rápido. [5]
- Siguen usando React en el servidor y lo precargan para las páginas siguientes del flujo de registro. [5]

Por qué es un caso relevante:

Muestra que React no es "todo o nada": una empresa grande puede decidir en qué parte de la aplicación conviene usarlo para equilibrar rendimiento y experiencia de usuario. [5]

Fuente:

- [5] Addy Osmani, "A Netflix Web Performance Case Study": https://medium.com/dev-channel/a-netflix-web-performance-case-study-c0bcde26a9d9

Fuentes

1. React, documentación oficial: https://react.dev/learn
2. React, "Start a New React Project": https://react.dev/learn/start-a-new-react-project
3. React Blog, "The React Foundation": https://react.dev/blog/2026/02/24/the-react-foundation
4. Engineering at Meta, "React at Meta Connect 2024": https://engineering.fb.com/2024/10/02/android/react-at-meta-connect-2024/
5. Addy Osmani, "A Netflix Web Performance Case Study": https://medium.com/dev-channel/a-netflix-web-performance-case-study-c0bcde26a9d9
6. React, "Meet the Team": https://react.dev/community/team
7. DEV Community, "The Untold Story of React": https://dev.to/visy_ani/the-untold-story-of-react-4bja
8. Linux Foundation, formación de la React Foundation: https://www.linuxfoundation.org/press/linux-foundation-announces-the-formation-of-the-react-foundation
9. Stack Overflow Developer Survey 2025: https://survey.stackoverflow.co/2025/
10. JavaScript Jabber 073, con Pete Hunt y Jordan Walke: https://topenddevs.com/podcasts/javascript-jabber/073-jsj-react-with-pete-hunt-and-jordan-walke